

Hands-on Lab : MySQL User Management, Access Control, and Encryption

Estimated time needed: 25 minutes

In this lab, first you will learn how to manage MySQL user accounts and roles using phpMyAdmin graphical user interface (GUI) tool. Then you will learn how to control access to MySQL databases and their objects. Finally you will learn how to secure your data adding extra layer of security using data encryption.

Objectives

After completing this lab, you will be able to use the phpMyAdmin to:

- Manage MySQL user accounts and roles
- Control access to MySQL databases and their objects
- Add last line of defense to secure data using encryption

Software Used in this Lab

In this lab, you will use [MySQL](#). MySQL is a Relational Database Management System (RDBMS) designed to efficiently store, manipulate, and retrieve data.



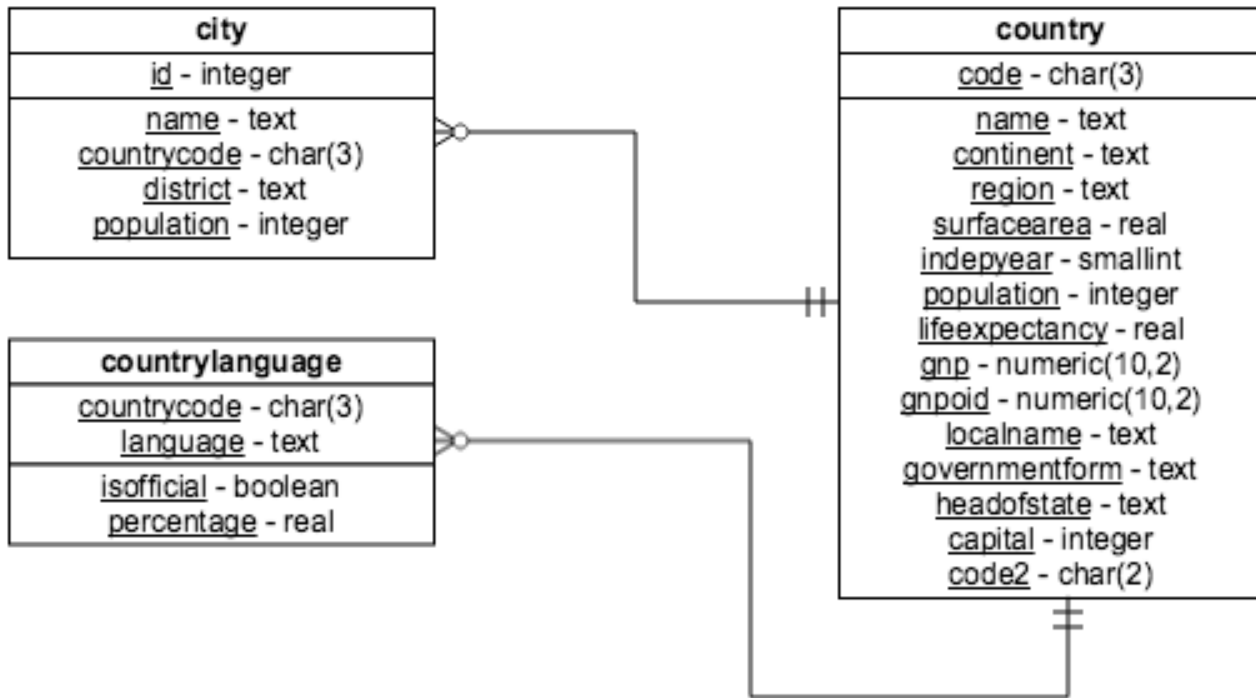
To complete this lab you will utilize the MySQL relational database service available as part of the IBM Skills Network Labs (SN Labs) Cloud IDE. SN Labs is a virtual lab environment used in this course.

Database Used in this Lab

The World database used in this lab comes from the following source: <https://dev.mysql.com/doc/world-setup/en/> under [CC BY 4.0 License](#) with [Copyright 2021 - Statistics Finland](#).

You will use a modified version of the database for the lab, so to follow the lab instructions successfully please use the database provided with the lab, rather than the database from the original source.

The following ERD diagram shows the schema of the World database:



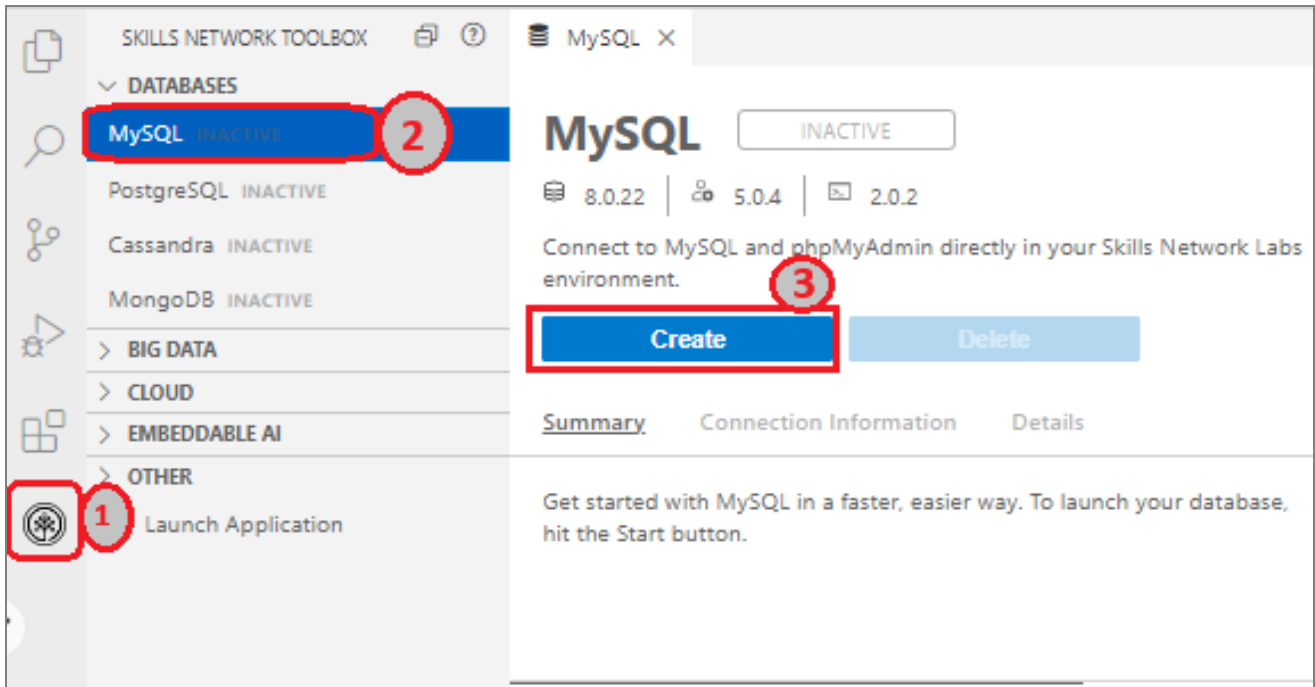
The first row is the table name, the second is the primary key, and the remaining items are any additional attributes.

Exercise 1: Manage MySQL user accounts and roles

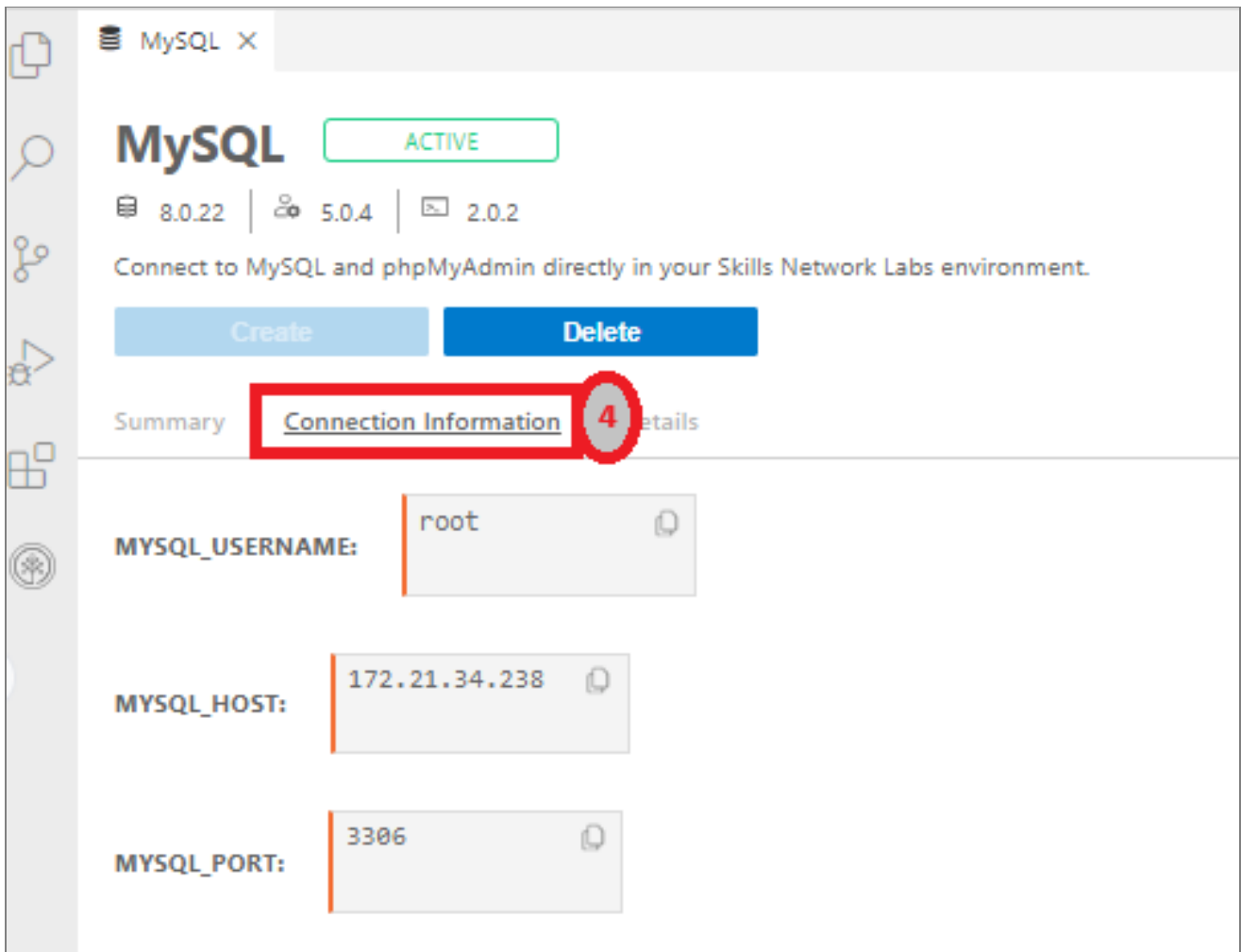
In this example exercise, you will go through an example on how to manage MySQL user accounts and roles using phpMyAdmin.

User management is the process of controlling which users are allowed to connect to the MySQL server and what permissions they have on each database. phpMyAdmin does not handle user management, rather it passes the username and password on to MySQL, which then determines whether a user is permitted to perform a particular action. Within phpMyAdmin, administrators have full control over creating users, viewing and editing privileges for existing users, and removing users.

1. Go to **Skills Network Toolbox** by clicking the following icon from the side by side launched Cloud IDE.
2. From the **Databases** drop-down menu, click **MySQL** to open the MySQL service session tab.
3. Click the **Create* button and wait until MySQL service session gets launched.



The MySQL server will take a few moments to start. Once it is ready, you will see the green “Active” label near the top of the window.



MySQL CLI Command:

MYSQL_COMMAND:

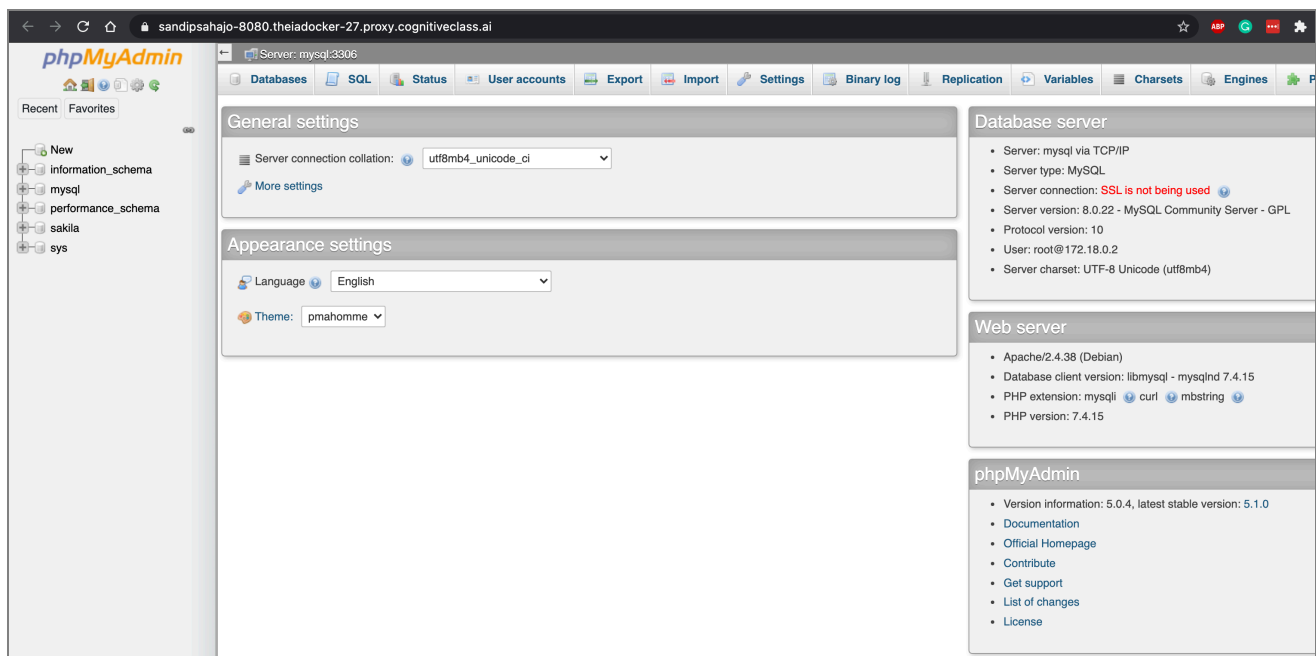
MYSQL_PASSWORD:

MYSQL_TITLE:

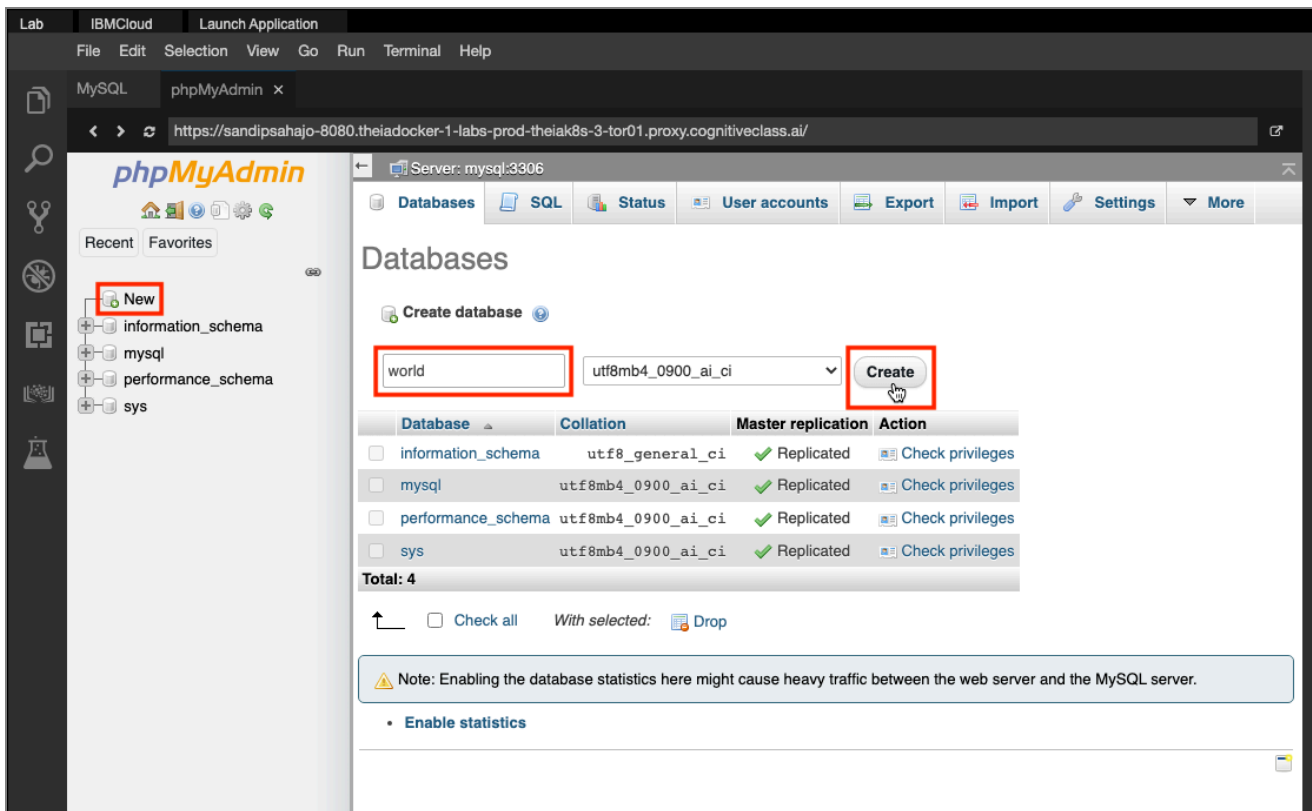
MYSQL_ID:

NOTE: Whenever you are required to enter your MySQL service session password from the MySQL service session tab at any step of the lab, copy the password by clicking on the small copy button on the right of the password block. Paste the password into the terminal using **Ctrl + V** (Mac: **⌘ + V**), and press **Enter** on the keyboard. For security reasons, you will not see the password as it is entered on the terminal.

3. Click **phpMyAdmin** button from the mysql service session tab. You will see the phpMyAdmin GUI tool.

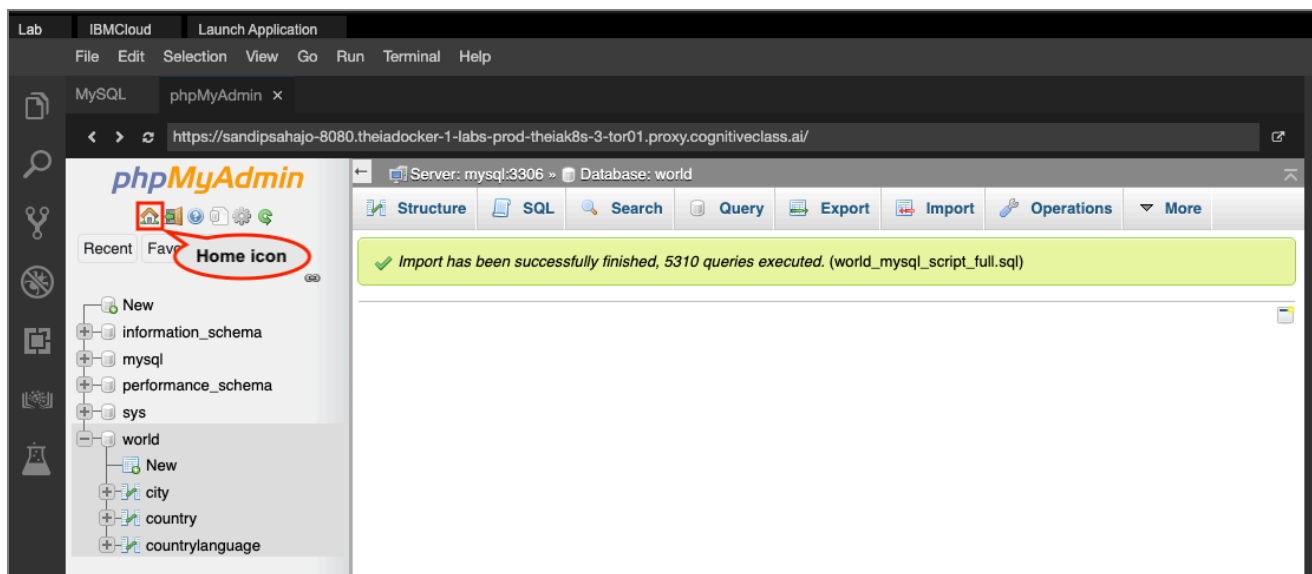
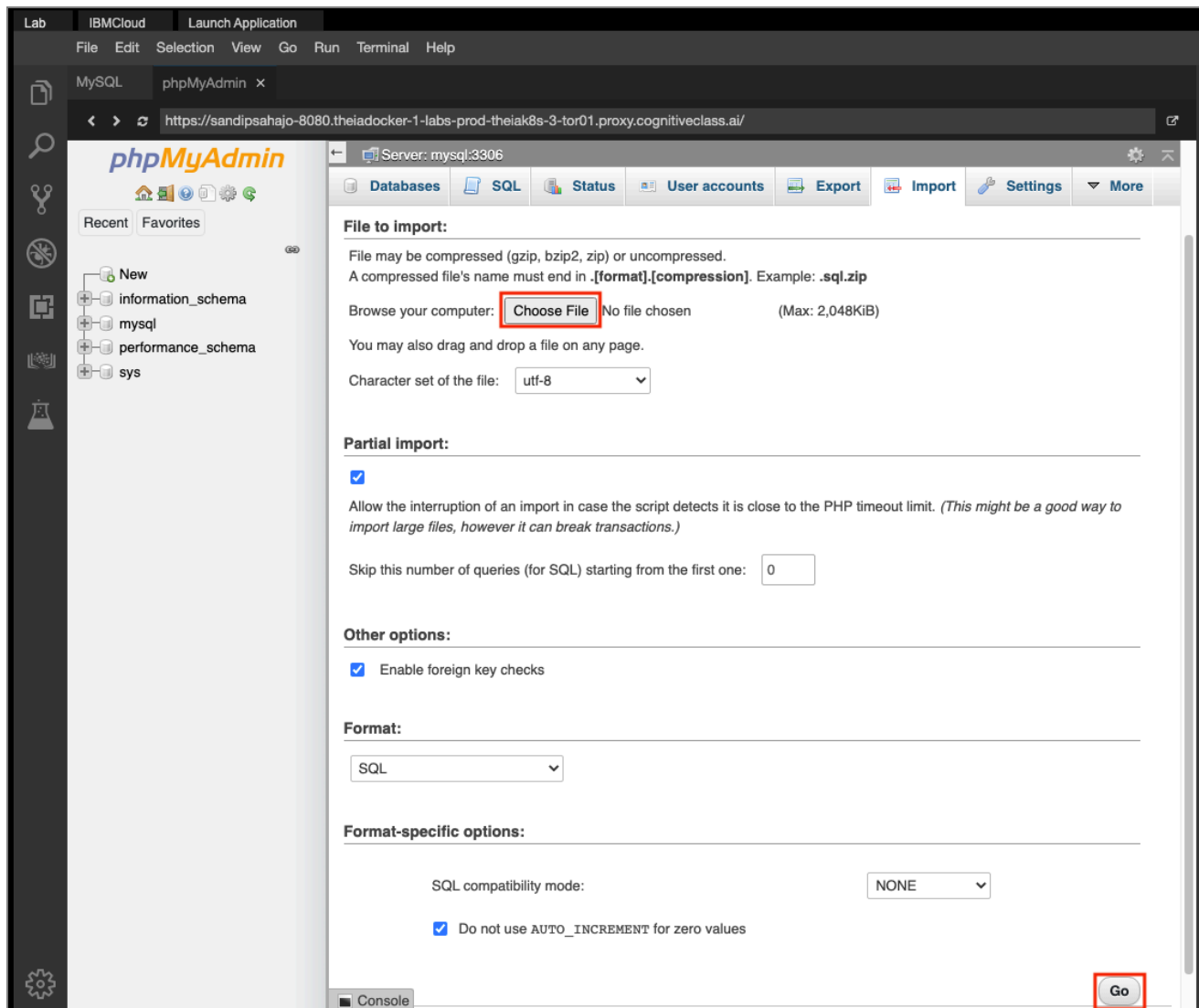


4. In the tree-view, click **New** to create a new empty database. Then enter **world** as the name of the database and click **Create**.



5. Go to the **Import** tab. Upload the following sql script file using the **Choose File** button (first download the following sql script file to your local computer storage). Then click **Go** button at the bottom. You will be notified when the import successfully gets finished. Click the **Home** icon.

- [world_mysql_script_full.sql](#)



6. Now you will create a user account with custom role “db_owner”. Usually a user with db_owner role has all global privileges and access to all existing databases. Go to the **User accounts** tab and click **Add user account**.

Lab IBMCloud Launch Application

File Edit Selection View Go Run Terminal Help

MySQL phpMyAdmin x

https://sandipsahajo-8080.theiadocker-1-labs-prod-theiak8s-3-tor01.proxy.cognitiveclass.ai/

Server: mysql:3306

Databases SQL Status **User accounts** Export Import Settings More

User accounts overview

	User name	Host name	Password	Global privileges	Grant	Action
☐	mysql.infoschema	localhost	Yes	SELECT	No	Edit privileges Export
☐	mysql.session	localhost	Yes	SHUTDOWN, SUPER	No	Edit privileges Export
☐	mysql.sys	localhost	Yes	USAGE	No	Edit privileges Export
☐	root	%	Yes	ALL PRIVILEGES	Yes	Edit privileges Export
☐	root	localhost	Yes	ALL PRIVILEGES	Yes	Edit privileges Export

☐ Check all With selected: [Export](#)

New

[Add user account](#)

[Remove selected user accounts](#)

(Revoke all active privileges from the users and delete them afterwards.)

☐ Drop the databases that have the same names as the users.

[Go](#)

Note: phpMyAdmin gets the users' privileges directly from MySQL's privilege tables. The content of these tables may differ from the privileges the server uses, if they have been changed manually. In this case, you should [reload the privileges](#) before you continue.

7. Fill the **Login Information** as shown in following image (enter your own password). Under **Global privileges**, click **Check all**. Scroll down and click **Go**.

Lab IBMCloud Launch Application

File Edit Selection View Go Run Terminal Help

MySQL phpMyAdmin x

https://sandipsahajo-8080.theiadocker-1-labs-prod-theiak8s-3-tor01.proxy.cognitiveclass.ai/

Server: mysql:3306

Databases SQL Status User accounts Export Import Settings More

Add user account

Login Information

User name: Use text field: db_owner

Host name: Local localhost

Password: Use text field: Strength: Extremely weak

Re-type:

Authentication Plugin: Caching sha2 authentication

Generate password: Generate

Database for user account

☐ Create database with same name and grant all privileges.

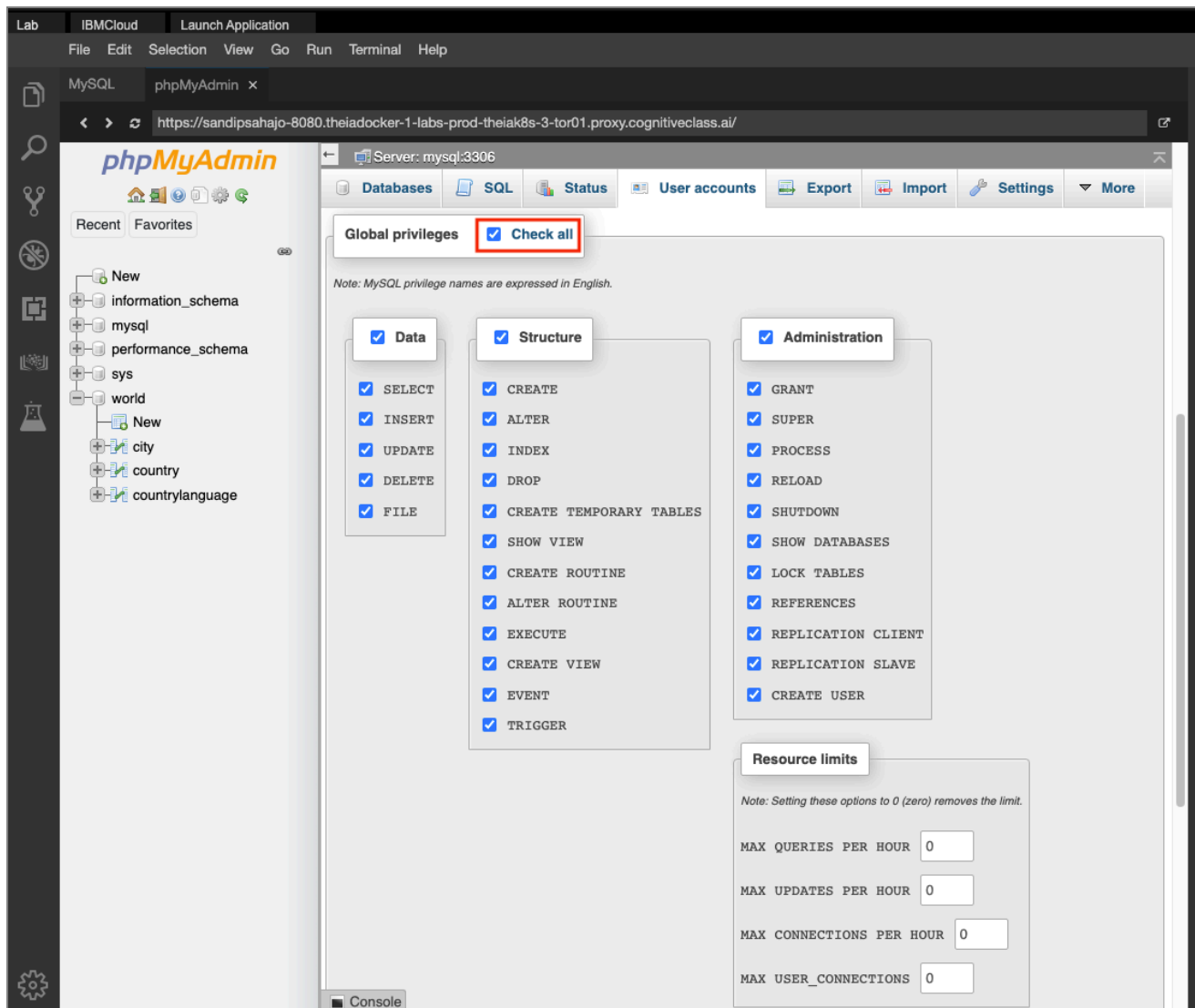
☐ Grant all privileges on wildcard name (username_%).

Global privileges ☐ Check all

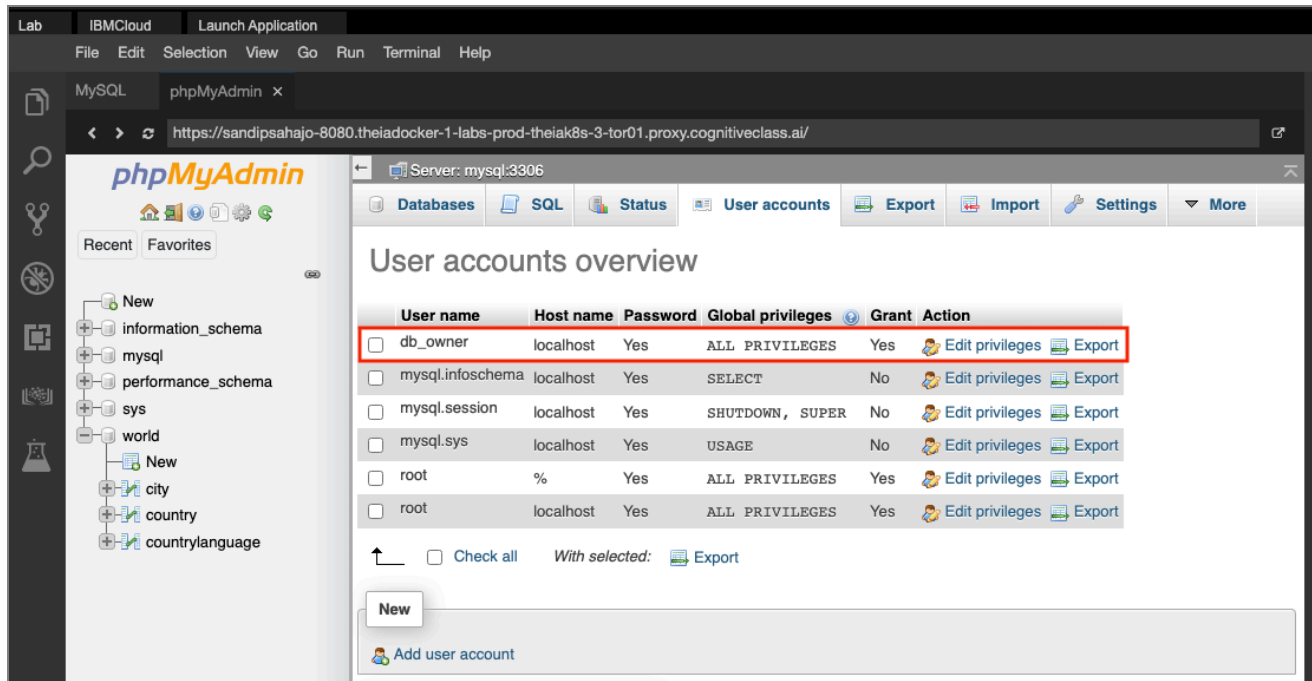
Note: MySQL privilege names are expressed in English.

☐ Data	☐ Structure	☐ Administration
☐ SELECT	☐ CREATE	☐ GRANT
☐ ALTER	☐ SUPER	

Console



8. You have successfully created a user account with appropriate privileges.

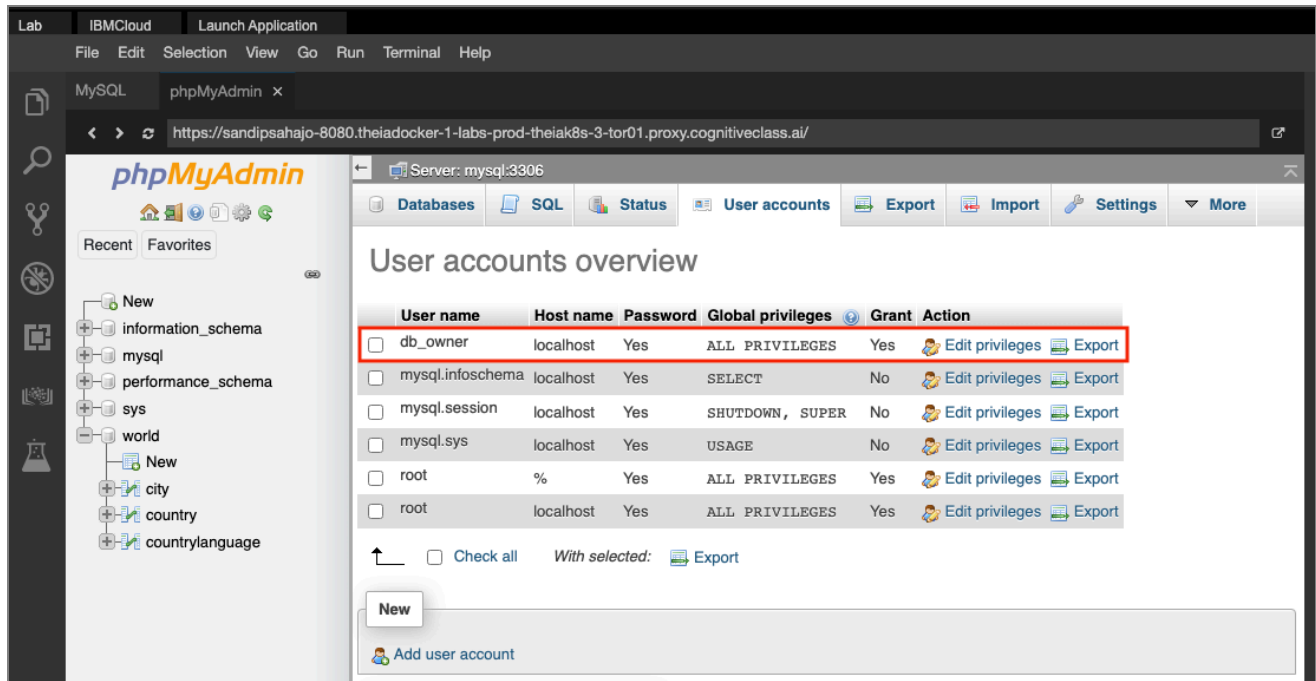


Exercise 2: Control access to MySQL databases and their objects

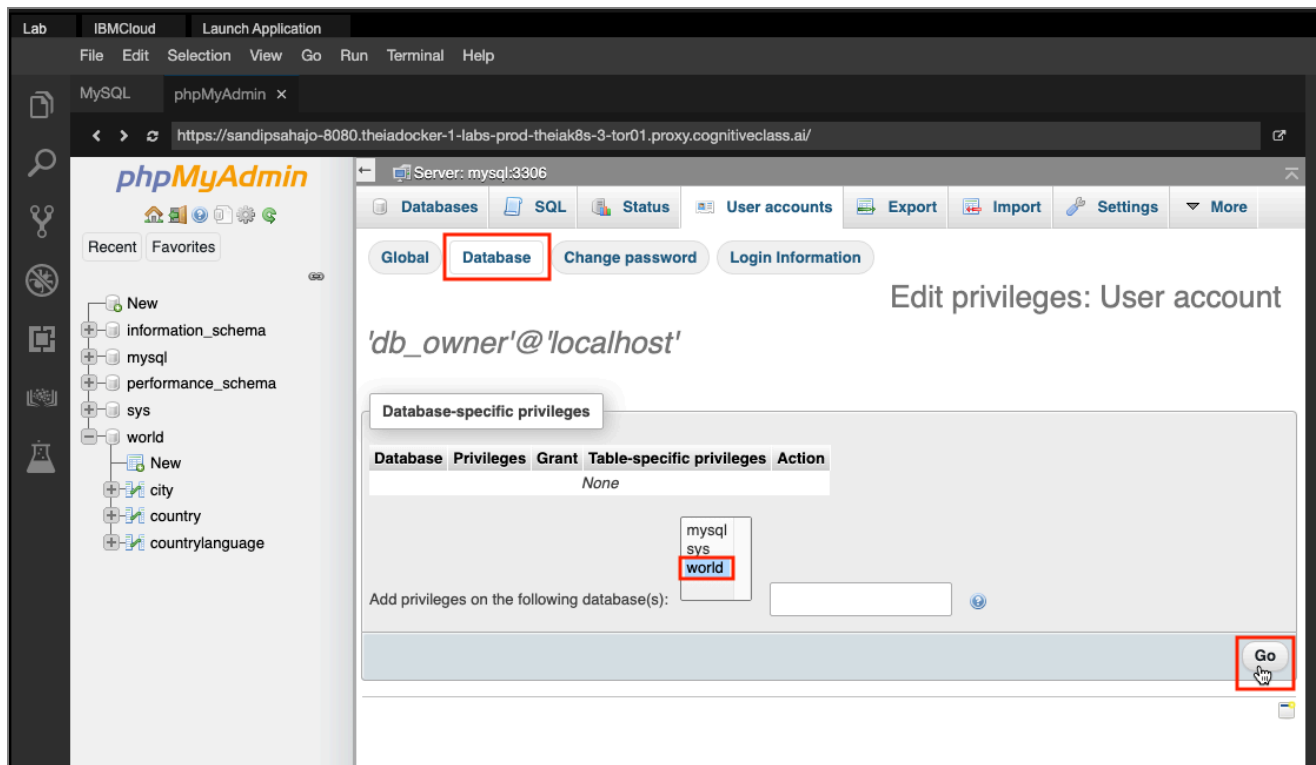
In this example exercise, you will learn how to control access to MySQL databases and their objects.

Making an exception to the user definition of db_owner role you created earlier, you will modify privileges of this user so that this user won't be able to update other columns except a specific column of a specific table of a specific database. You will restrict db_owner from updating all the other columns except the column **Population** of the table **city** of the database **world**.

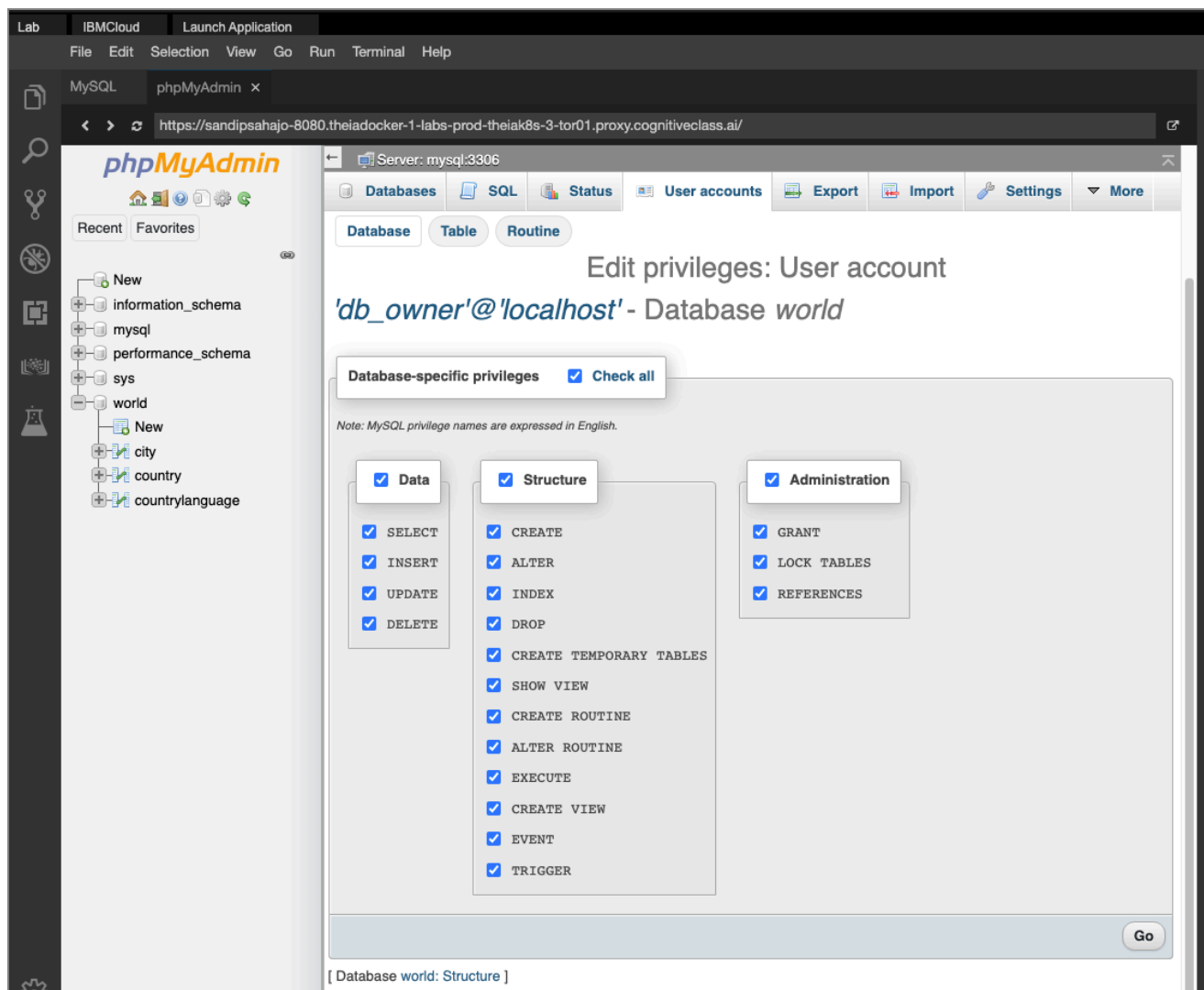
1. Go to **Home > User accounts** tab. Click **Edit privileges** option of **db_owner** user name.



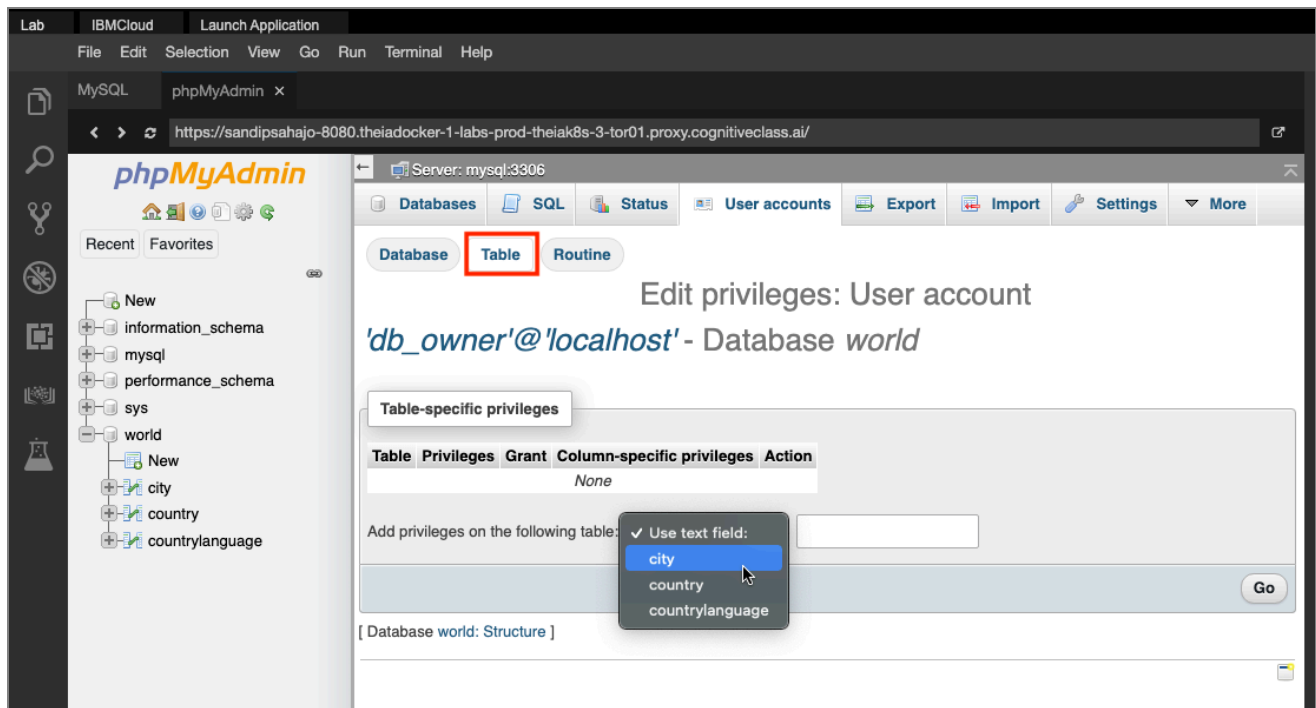
2. Under **Database** sub-tab, select **world** database and click **Go**.



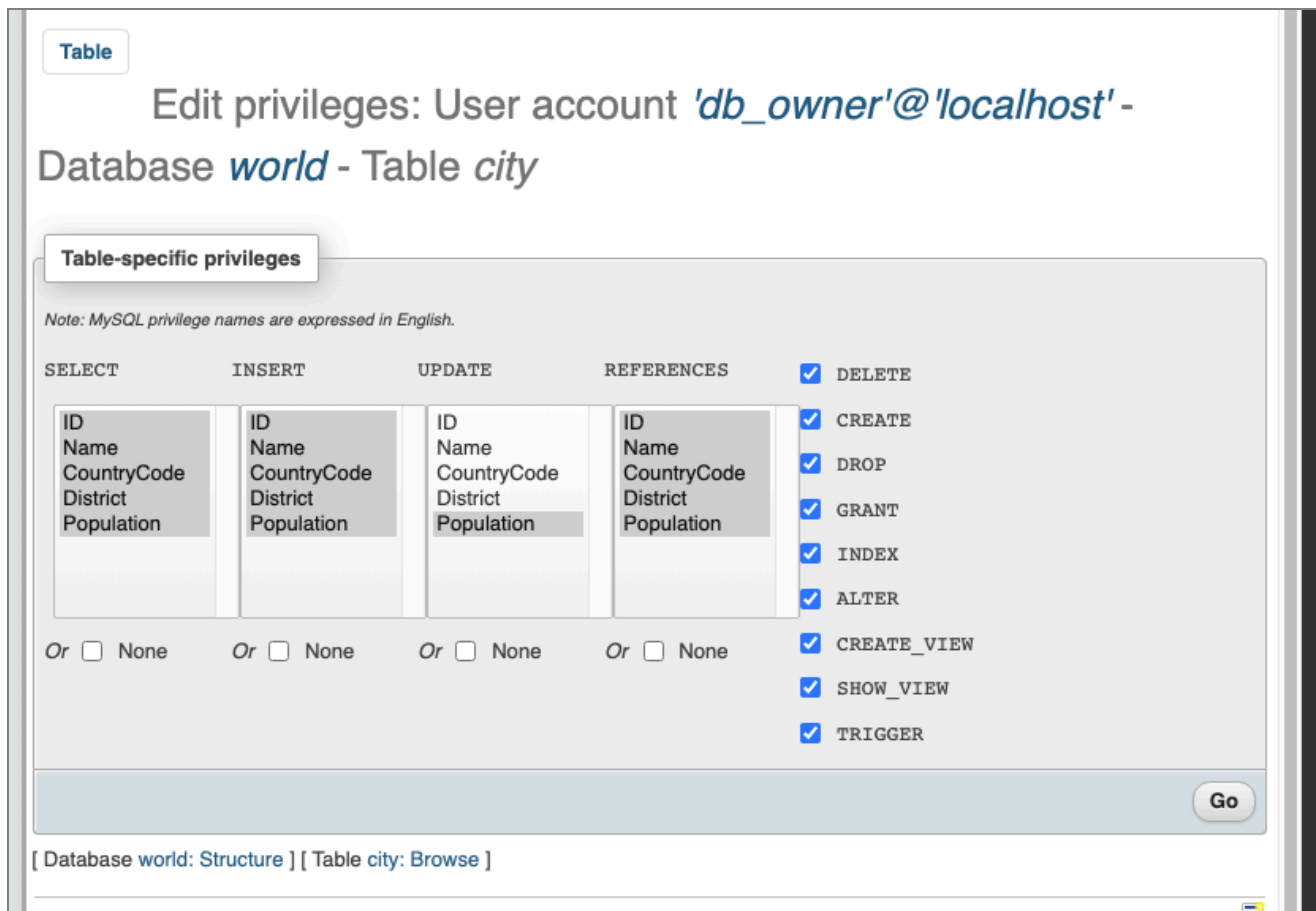
3. Under **Database-specific privileges**, select **Check all** and click **Go** at the bottom.



4. Switch to **Table** sub-tab. Select the table **city** from the drop-down menu and click **Go**.



5. Under **Table-specific privileges**, configure all the SQL commands and their custom access to the columns of the table **city** as shown below. Then click **Go**. Such table-specific privilege configuration will restrict **db_owner** from updating all the other columns except the column **Population** of the table **city** of the database **world**.

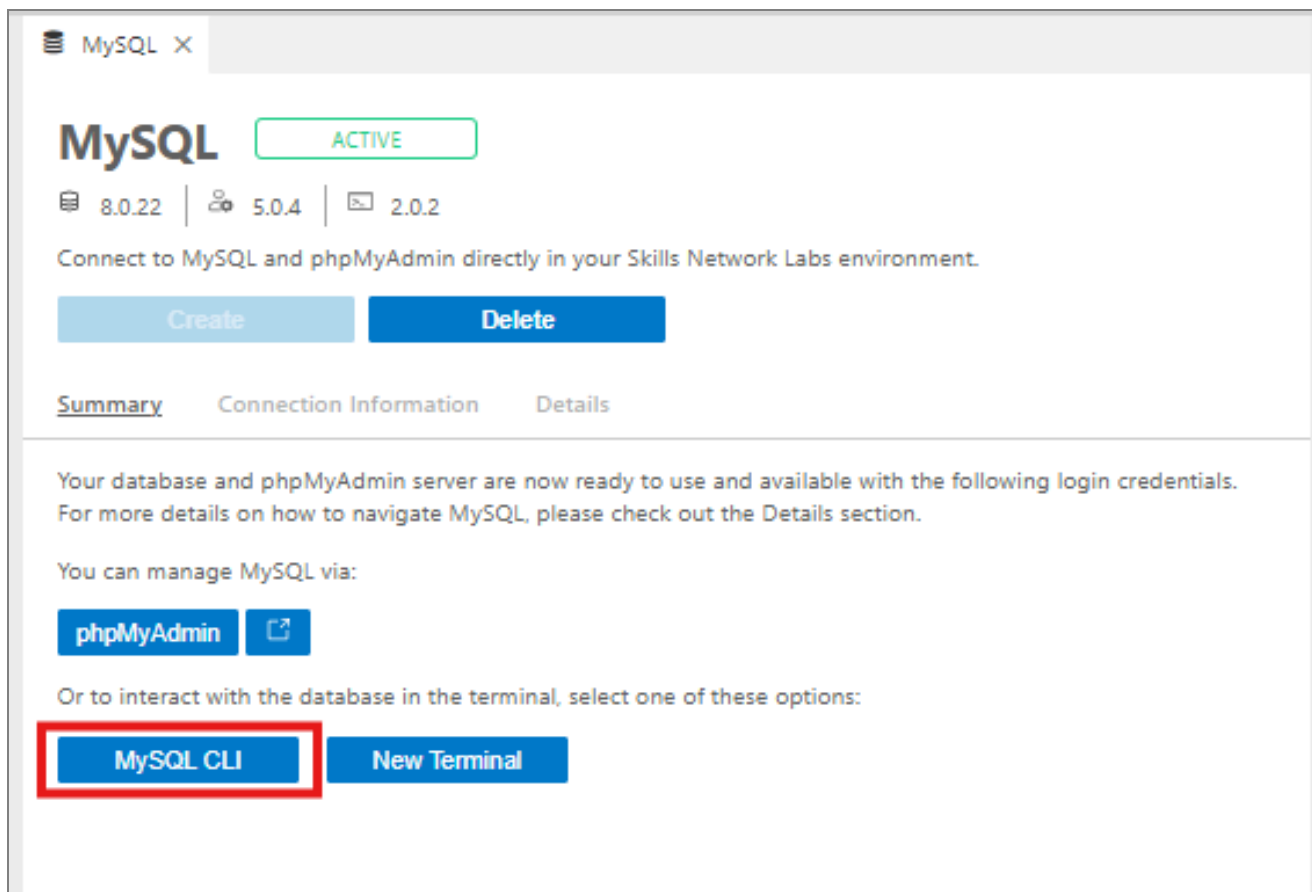


Exercise 3: Secure data using encryption

In this example exercise, you will learn how to secure your data adding extra layer of security using data encryption. There may be certain parts of your database containing sensitive information which *should not* be stored in plain text. This is where encryption comes in.

You will implement encryption and decryption of a column in the world database using the official AES (Advanced Encryption Standard) algorithm. AES is a symmetric encryption where the same key is used to encrypt and decrypt the data. The AES standard permits various key lengths. By default, key length of 128-bits is used. Key lengths of 196 or 256 bits can be used. The key length is a trade off between performance and security. Let's get started.

1. Click the **MySQL CLI** button from the mysql service session tab.



2. First, you will need to hash your passphrase (consider your passphrase is **My secret passphrase**) with a specific hash length (consider your hash length is **512**) using a hash function (here you will use hash function from **SHA-2** family). It is good practice to hash the passphrase you use, since storing the passphrase in plaintext is a significant security vulnerability. Use the following command in the terminal to use the SHA2 algorithm to hash your passphrase and assign it to the variable `key_str`:

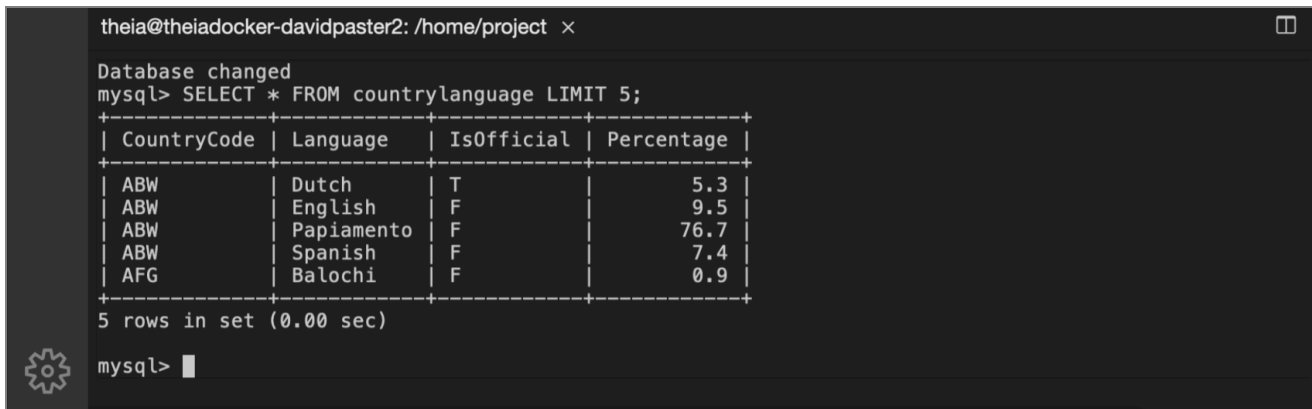
```
SET @key_str = SHA2('My secret passphrase', 512);
```

3. Now, let's take a look at the `world` database. First, you will want to connect to the database by entering the following command in the CLI:

```
USE world;
```

4. Next, let's take a quick look at the `countrylanguage` table in our database with the following command:

```
SELECT * FROM countrylanguage LIMIT 5;
```



The screenshot shows a terminal window with the title `theia@theiadocker-davidpaster2: /home/project`. The prompt is `mysql>`. The command entered is `SELECT * FROM countrylanguage LIMIT 5;`. The output is a table with 4 columns: `CountryCode`, `Language`, `IsOfficial`, and `Percentage`. The data rows are:

CountryCode	Language	IsOfficial	Percentage
ABW	Dutch	T	5.3
ABW	English	F	9.5
ABW	Papiamentu	F	76.7
ABW	Spanish	F	7.4
AFG	Balochi	F	0.9

Below the table, it says `5 rows in set (0.00 sec)`. The prompt `mysql>` is visible at the bottom.

For *demonstration purposes*, **suppose** that the last column in the table, labeled *Percentage* contains sensitive data, such as a citizen's passport number. Storing such sensitive data in plain text is an enormous security concern, so let's go ahead and encrypt that column.

5. To encrypt the *Percentage* column, we will first want to convert the data in the column into binary byte strings of length 255 by entering the following command into the CLI:

```
ALTER TABLE countrylanguage MODIFY COLUMN Percentage varbinary(255);
```

6. Now to actually encrypt the *Percentage* column, we use the AES encryption standard and our hashed passphrase to execute the following command:

```
UPDATE countrylanguage SET Percentage = AES_ENCRYPT(Percentage, @key_str);
```

7. Let's go ahead and see if the column was successfully encrypted by taking another look at the *countrylanguage* table. We again run the same command as in step 4:

```
SELECT * FROM countrylanguage LIMIT 5;
```

```
theia@theiadocker-davidpaster2: /home/project ×

mysql> SELECT * FROM countrylanguage LIMIT 5;
+-----+-----+-----+-----+
| CountryCode | Language | IsOfficial | Percentage |
+-----+-----+-----+-----+
| ABW         | Dutch   | T          | d0L0D00e0!0h~ |
| ABW         | English | F          | 0rG0000000(0R0KK |
| ABW         | Papiamentto | F        | 100A0;00.z0m |
| ABW         | Spanish | F          | K0000A0 o00 |
| AFG         | Balochi | F          | 00'0hB0zI0Hr |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

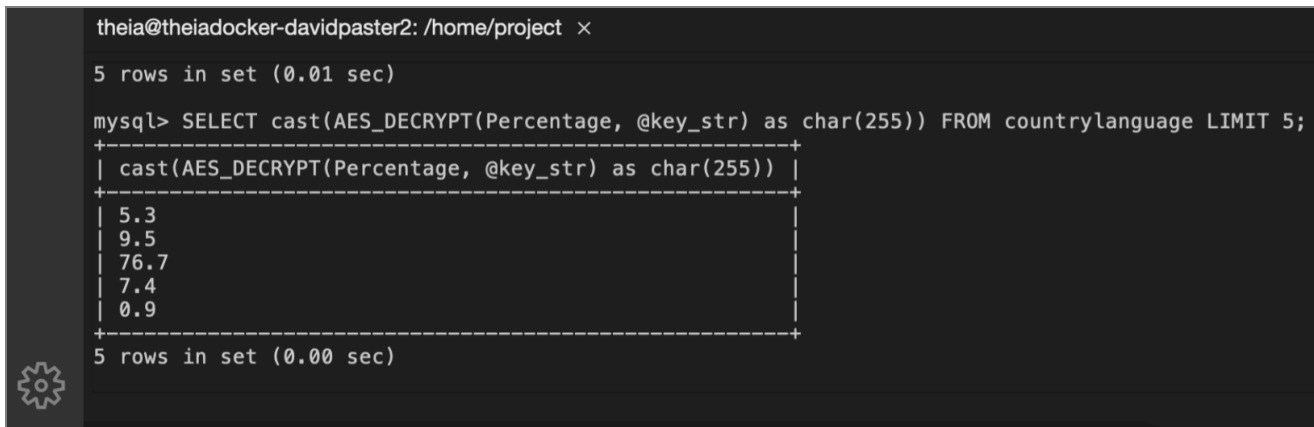
mysql> █
```

As you can see, the data on the *Percentage* column is encrypted and completely illegible.

8. The supposedly sensitive data is now encrypted and secured from prying eyes. However, we should still have a way to access the encrypted data when needed. To do this, we use the `AES_DECRYPT` command, and since AES is symmetric, we use the same key for both encryption and decryption. In our case, recall that

the key was a passphrase which was hashed and stored in the variable `key_str`. Suppose we need to access the sensitive data in that column. We can do so by entering the following command in the CLI:

```
SELECT cast(AES_DECRYPT(Percentage, @key_str) as char(255)) FROM countrylanguage;
```



The screenshot shows a terminal window with the prompt `theia@theiadocker-davidpaster2: /home/project`. It displays the execution of a MySQL query: `mysql> SELECT cast(AES_DECRYPT(Percentage, @key_str) as char(255)) FROM countrylanguage LIMIT 5;`. The output is a table with one column, `cast(AES_DECRYPT(Percentage, @key_str) as char(255))`, containing five rows of values: 5.3, 9.5, 76.7, 7.4, and 0.9. The terminal also shows the execution time for both the command and the result set.

```
theia@theiadocker-davidpaster2: /home/project ×  
5 rows in set (0.01 sec)  
  
mysql> SELECT cast(AES_DECRYPT(Percentage, @key_str) as char(255)) FROM countrylanguage LIMIT 5;  
+-----+  
| cast(AES_DECRYPT(Percentage, @key_str) as char(255)) |  
+-----+  
| 5.3 |  
| 9.5 |  
| 76.7 |  
| 7.4 |  
| 0.9 |  
+-----+  
5 rows in set (0.00 sec)
```

Practice Exercise: Control access to MySQL databases and their objects

In this practice exercise, you will get to put what you learned to use and modify privileges for a user.

*Scenario: You will modify privileges of the user **db_owner** you created in example exercise A such a way that this user won't be able to insert, update and delete any column of a specific table **country** of the **world** database.*

- ▶ [Hint \(Click Here\)](#)
- ▶ [Solution \(Click Here\)](#)

Summary

In this lab, first learned how to manage MySQL user accounts and roles using phpMyAdmin graphical user interface (GUI) tool. You also learned how to control access to MySQL databases and their objects. Finally, you learned how to secure your data adding extra layer of security using data encryption.

Congratulations! You have completed this lab, and you are ready for the next topic.

Author(s)

- [Sandip Saha Joy](#)

Other Contributor(s)

- [David Pasternak](#)

© IBM Corporation 2023. All rights reserved.